DocEntry-AI

AI-Powered Document Data Extraction for the Import & Export Industry

PROJECT REPORT

Submitted by	Prajwal Shinde
Project type	College project · individual
Academic year	2026–27
Technology	Python · Streamlit · Google Gemini API
Repository	https://github.com/prajwalshinde752-nonveg/DocEntry-AI
Live demo	Streamlit Community Cloud

Contents

1. Abstract
2. Introduction and Background
3. Problem Statement
4. Objectives
5. Proposed Solution and Scope
6. System Architecture
7. Technology Stack
8. Implementation
9. Workflow and Supported Inputs
10. Results
11. Testing
12. Expected Outcome
13. Security and Privacy
14. Limitations
15. Future Scope
16. Conclusion
17. How to Run and Deploy

References

Appendix A — Field Schemas

1. Abstract

Every import and export shipment produces a trail of trade documents — tax invoices, shipping bills, bills of entry, purchase orders and delivery challans. Customs compliance, DGFT scheme reconciliation, GST filing and bank submissions all require the contents of those documents to be recorded in Excel registers, and today that transfer is done by hand: one operator reads a PDF on one screen and re-types each value into a spreadsheet on the other. It is slow, repetitive and prone to transposed digits, lost leading zeros and values placed in the wrong column.

DocEntry-AI removes the typing step. A user uploads one or many documents — a digital PDF, a phone photo or scan, a Word file, a CSV export, or plain text pasted directly into the app — chooses the columns required for the register, and the application reads the document with a multimodal large language model, maps the values to a document-specific schema, and presents them in an editable grid. One click exports the approved rows to a fresh Excel file or appends them to an existing master register.

The application is written entirely in Python and uses Streamlit for the interface, PyMuPDF, Pillow and python-docx for document handling, the OpenAI-compatible Gemini API for extraction, and openpyxl for Excel output. The operator's role shifts from data entry to data review: instead of producing the data, they confirm it.

2. Introduction and Background

A single export consignment can generate a dozen documents: a commercial invoice, a packing list, a shipping bill, a bill of lading, a bank realisation certificate and supporting declarations. Each arrives as a PDF or a scan from a different source — a customs portal, a freight forwarder, a bank, an overseas buyer — and each has its own layout. There is no shared template, so a fixed extraction rule or a simple copy-paste workflow does not survive contact with real documents.

The system of record in most trading houses is nevertheless simple: an Excel register. Auditors, customs officers and bank officials ask for that register, not the original PDFs. The gap between the two is currently bridged by a person reading and retyping.

> *The bottleneck is not a lack of data. It is the manual effort required to move data from a document into a structured row.*

3. Problem Statement

Import and export businesses still depend on manual data entry to move information from trade documents into Excel. The process is slow, repetitive and error-prone, it does not scale with shipment volume, and the resulting registers are unreliable precisely where accuracy matters most — customs compliance, scheme reconciliation and audit.

3.1 Current process

Step	How it is done today
1. Receive	Documents arrive as PDFs or scanned images, in many different layouts.
2. Read	An operator manually locates each required field inside the document.
3. Type	Each value is keyed into the matching Excel column by hand.
4. Repeat	The same cycle repeats for every document, every day.
5. Verify	A second person re-checks entries, or errors surface later during audit.

3.2 Pain points

- **Time consuming.** A single multi-item document can take 15–30 minutes to key in.
- **High error rate.** Typing introduces transposed digits, stripped leading zeros, values in the wrong column and misread scans.
- **Costly consequences.** A wrong invoice value, IGST amount or licence number surfaces later as a customs query, a rejected claim or a reconciliation mismatch.
- **Does not scale.** More shipments means proportionally more operators; experience does not increase throughput.
- **Format fragmentation.** Every port, bank and buyer issues documents in a different layout, so no fixed template works.
- **Low-value work.** Skilled documentation staff spend the day typing instead of checking compliance.

4. Objectives

1. Accept any common document format an exporter actually receives: digital PDF, scanned PDF, phone photo, Word file or pasted text.
2. Extract the required fields with a free multimodal language model, without OCR templates or fixed coordinates.
3. Support several trade document types through per-type field schemas that can be extended by editing one dictionary.
4. Produce one Excel row per line item, so a multi-item invoice becomes a usable register rather than a single summary row.
5. Keep a human review step before export, so the operator confirms values instead of trusting them blindly.
6. Export to a new register or append to an existing master file, and run with zero infrastructure cost.

5. Proposed Solution and Scope

DocEntry-AI is a single Python web application. The user adds documents, selects the columns wanted in the register, reviews the extracted values and downloads Excel. The AI is used only for reading and field matching; all file handling, validation and Excel writing is ordinary Python, which keeps the output predictable.

In scope	Out of scope (first version)
Extraction from tax invoices, shipping bills, purchase orders, bills of entry and delivery challans	Direct filing to customs, DGFT or banking portals
Per-type column schemas with user-selectable columns	Replacing accounting or ERP software
Digital PDFs, scanned PDFs, photos, Word, TXT, CSV and pasted text	Legal or regulatory interpretation of document contents
One row per line item, editable before export	Handwritten documents and non-Latin scripts
Fresh Excel register or append to an existing one	Multi-user accounts, database storage or an audit trail

6. System Architecture

The application is deliberately small: five Python modules and one external API. The diagram below shows the path a document takes from upload to Excel.

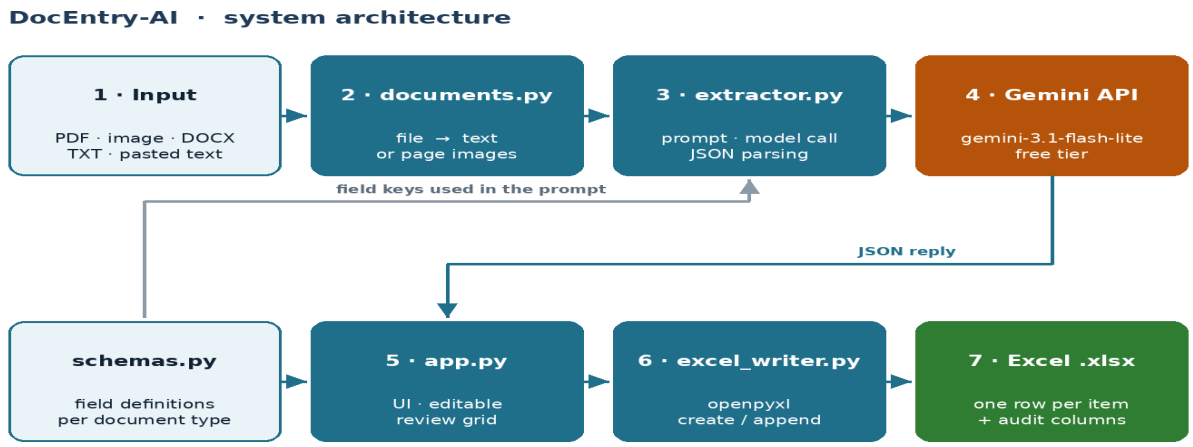


Figure 1 — Document flow through DocEntry-AI

6.1 Path of one document

1. Read. documents.py converts the uploaded file into text, or into JPEG images when the content is a scan or a photo.
2. Prompt. extractor.py combines the document type, the selected field keys and the document content into one prompt.
3. Extract. The prompt is sent to gemini-3.1-flash-lite through the OpenAI-compatible endpoint. The model returns JSON only.
4. Parse. The reply is cleaned, parsed and normalised so every requested key exists with a confidence level.
5. Review. app.py shows one editable table row per line item. Nothing is written until the user approves.
6. Export. excel_writer.py writes a new workbook or appends rows below the existing data of a master file.

6.2 Design decisions

- **Text first, images only when needed.** A digital PDF is sent as text because it is cheaper and more accurate than rendering pages. Pages are rendered only when the PDF has almost no extractable text.
- **One request per document.** Batch handling lives in the app loop, so one failure does not abort the rest of the batch.
- **JSON contract.** The model is instructed to return a fixed JSON shape, which removes guesswork from parsing.
- **Schema-driven columns.** Adding a new document type means adding one entry to DOCUMENT_TYPES in schemas.py; no other file changes.

7. Technology Stack

Python is the only programming language used. There is no HTML, CSS or JavaScript written by hand — Streamlit generates the page from Python.

Layer	Technology	Purpose in this project
Language	Python 3.12 (cloud), 3.14 (local)	All application logic
Interface	Streamlit	Sidebar, tabs, uploader, editable table, download button
AI model	Google Gemini — gemini-3.1-flash-lite	Reads documents and returns field values as JSON
AI client	openai Python SDK	Calls Gemini's OpenAI-compatible endpoint
PDF	PyMuPDF	Text extraction; rendering scanned pages to images
Images	Pillow	Opens photos, applies EXIF rotation, downscales to 1600 px
Word	python-docx	Reads paragraphs and tables from .docx files
Excel	openpyxl	Creates and appends to .xlsx registers, with header styling
Config	python-dotenv	Loads API settings from .env locally
Hosting	Streamlit Community Cloud	Free public deployment from GitHub

The free tier of the AI model was a hard requirement, so the model was chosen after timing real requests: gemini-3.1-flash-lite answers in roughly 2–4 seconds per document, where a larger model either timed out or was unavailable to new accounts.

8. Implementation

The project contains five Python modules totalling about 1,020 lines, plus configuration and sample documents.

File	Lines	Responsibility
app.py	453	All UI: sidebar, the four steps, tabs, progress, review grid, download
extractor.py	232	Prompt construction, model call, JSON parsing, error reporting
documents.py	150	Any supported file → text or images
schemas.py	101	Field definitions for the five document types

File	Lines	Responsibility
excel_writer.py	84	Workbook creation and append-to-master logic

8.1 documents.py — one reader for every format

Each file type is converted into a small Document object holding either text or a list of JPEG images. A PDF is opened with PyMuPDF and its text is joined across pages. If fewer than 40 characters are found, the file is treated as a scan: the first five pages are rendered at 150 dpi and re-encoded as JPEG. Photos are EXIF-rotated, converted to RGB and downscaled to 1600 px wide, which keeps phone photos within a sensible request size. Word files are read paragraph by paragraph, including table cells joined with pipe characters. A CSV export is read with Python's csv module, auto-detecting comma, semicolon or tab separators, and is written out one pipe-separated line per row so the model sees the header and the values together. Files larger than 200 rows are cut at the 200th row, with a note shown in the preview.

8.2 extractor.py — prompt, call, parse

The prompt states the document type, the exact field keys and labels, and the required JSON shape. The reply is parsed leniently: code fences are stripped, and if the model wraps the JSON in extra text, the outermost braces are recovered. Both the current {"records": [...]} shape and the older {"fields": {...}} shape are accepted, so an old cached module cannot break the page.

```
{
  "records": [
    {"fields": {"invoice_no": "INV/2026/0100", "hsn_code": "5208", "quantity": "500"},
     "confidence": {"invoice_no": "high", "hsn_code": "high", "quantity": "high"}}
  ]
}
```

If the provider rejects JSON mode or returns a 400 that is not an authentication problem, the request is retried once without response_format. Every requested key is guaranteed to exist in the result, with an empty value and low confidence when the document does not contain it.

8.3 schemas.py — the column contract

DOCUMENT_TYPES maps each document type to a list of (key, label) pairs. The keys are used in the prompt and in Excel; the labels are what the user sees. Two audit columns, Source File and Extracted At, are appended automatically so every exported row can be traced back to its document.

8.4 excel_writer.py — the register

Rows are written with openpyxl. A fresh workbook gets a styled header row, a frozen top pane and sensible column widths. When a master file is supplied, the existing header is inspected, missing columns are added, and new rows are appended below the last used row without disturbing the existing data.

8.5 app.py — the four-step interface

The page is a linear flow: add documents, choose columns, extract and review, download. The upload tab and the paste-text tab feed the same list of sources, so a pasted document is processed exactly like a file. Extraction runs in a loop with a progress bar; each line item becomes a row. Low-confidence fields are listed under the table so the operator knows what to check first.

9. Workflow and Supported Inputs

Step	What the user does	What the application does
1 Add documents	Uploads one or many files, or pastes text	Queues each source and shows a preview of the first one
2 Choose columns	Selects the columns needed in the register	Maps labels to schema keys for the chosen document type
3 Extract and review	Presses Extract fields, then corrects cells	Calls the model, creates one row per line item, flags low confidence
4 Download	Optionally uploads an existing master register	Builds or appends the workbook and offers it as a download

9.1 Supported input formats

Input	How it is read	Best for
Digital PDF	Text extracted directly with PyMuPDF	Portal-generated invoices and bills
Scanned PDF	First 5 pages rendered to JPEG and read visually	Scanned customs paperwork
JPG / PNG / WEBP / BMP / TIFF	Pillow load, EXIF rotation, downscale to 1600 px	Phone photos of documents
DOCX	Paragraphs and table cells via python-docx	Documents received in Word
TXT	Decoded as UTF-8 with replacement for bad bytes	Exported text from other systems
CSV	Parsed with Python's csv module; comma, semicolon or tab separated	Exports from Excel, Tally or a portal
Pasted text	Used as-is, with a user-supplied row name	Emails and portal pages

9.2 Supported document types

Document type	Fields
Tax Invoice	19 fields — invoice no, date, seller/buyer, GSTIN, place of supply, HSN, description, quantity, unit, rate, taxable value, IGST, CGST, SGST, total, currency
Shipping Bill	17 fields — bill no, date, exporter, IEC, GSTIN, invoice, ports, vessel, HSN, description, quantity, FOB value, currency, scheme
Purchase Order	12 fields — PO no, date, buyer, supplier, HSN, description, quantity, unit, rate, total, currency, delivery terms
Bill of Entry	14 fields — BE no, date, importer, IEC, GSTIN, invoice, ports, HSN, description, quantity, assessable value, duty, currency
Delivery Challan	10 fields — challan no, date, consignor, consignee, HSN, description, quantity, unit, value, vehicle no

10. Results

The same test invoice was supplied in three different formats and produced identical values, confirming that the text path and the vision path agree.

Field	Text PDF	Scanned JPG	DOCX
Invoice number	INV/2026/0042	INV/2026/0042	INV/2026/0042

Field	Text PDF	Scanned JPG	DOCX
Invoice date	15-10-2026	15-10-2026	15-10-2026
Seller GSTIN	27ABCDE1234F1Z5	27ABCDE1234F1Z5	27ABCDE1234F1Z5
HSN code	5208	5208	5208
Quantity	1500	1500	1500
Total invoice value	371700.00	371700.00	371700.00

10.1 Multi-item extraction

A three-item invoice (INV/2026/0100) produced three rows in one pass, each carrying the invoice-level values so it can stand alone in the register.

Row	HSN	Description	Quantity	Rate	Invoice no.
1	5208	Cotton Fabric 40s	500	238.00	INV/2026/0100
2	5205	Cotton Yarn 20s	300	150.00	INV/2026/0100
3	5402	Polyester Thread	200	210.00	INV/2026/0100

10.2 Timing

Operation	Measured time
Text PDF → fields	2.3 – 2.4 s
Scanned image → fields	4.3 s
DOCX → fields	2.3 s
Connection test (one short reply)	about 1 s

11. Testing

#	Test case	Input	Expected	Result
1	Digital PDF, single item	sample_tax_invoice.pdf	1 row, all fields	Pass
2	Scanned image	sample_tax_invoice_scan.jpg	1 row, values match PDF	Pass
3	Word document	sample_tax_invoice.docx	1 row, values match PDF	Pass
4	Pasted text	Invoice text pasted in the tab	1 row named by the user	Pass
5	Multi-item invoice	sample_tax_invoice_multi.pdf	3 rows, 3 products	Pass
6	Batch upload	PDF + JPG + DOCX together	3 rows in one register	Pass
7	CSV export, three line items	sample_tax_invoice.csv	3 rows, values match the file	Pass
8	Invalid API key	Wrong key in secrets	Clear error, no crash	Pass
9	Unsupported file	e.g. .xlsx used as input	Readable message, file skipped	Pass

The interface was also exercised headlessly with Streamlit's AppTest: text was placed in the paste box, the extract button was pressed, and the resulting session rows were checked programmatically. That test is what caught the original one-row-only behaviour.

12. Expected Outcome

Parameter	Manual process	With DocEntry-AI
Time per document	15–30 minutes	Seconds
Error rate	High — typing and transposition	Low — extraction plus human review
Scalability	More volume needs more staff	Batch processing with the same staff
Staff role	Data entry	Verification and compliance
Auditability	No trace of who typed what	Source file and timestamp on every row

13. Security and Privacy

- **Key handling.** The API key lives in .env locally and in Streamlit Cloud Secrets when deployed. Both are excluded from Git, so the public repository contains no credentials.
- **No storage.** Uploaded documents are processed in memory and are not written to disk or to a database.
- **Provider exposure.** Document text or page images are sent to Google's Gemini API for extraction; this is the only third party involved.
- **Review before export.** Nothing is committed to a register until the user presses download.

14. Limitations

- One document type per run: all files in a batch must belong to the type selected in the sidebar.
- Scanned PDFs: only the first five pages are read, to control request size.
- CSV files: only the first 200 rows are read.
- Multi-item documents are capped at 50 rows per document.
- Photos are downscaled to 1600 px, which is enough for print but not for very small print.
- Handwritten text, stamps and non-Latin scripts are not reliably read.
- Values are copied, not validated: GSTIN checksums, date arithmetic and total reconciliation are not checked.
- The free tier has request-rate limits, so very large batches may need to be split.

15. Future Scope

- Automatic document-type detection, so a mixed batch no longer needs the sidebar selection.
- Splitting a scanned batch PDF into separate documents by page.
- Validation rules: GSTIN checksum, quantity × rate = taxable value, tax arithmetic, date sanity.

- Persistent master register with a database and a searchable history.
- Highlighting low-confidence cells directly in the Excel file.
- Multi-language OCR and handwriting support.
- User accounts, roles and a full audit trail.
- Direct export connectors for accounting software and ERP systems.

16. Conclusion

The bottleneck in trade documentation is not a lack of data but the manual effort required to transfer it. DocEntry-AI removes that effort: documents of any common format are read by a multimodal model, mapped to a per-type schema, reviewed in an editable grid, and written to an Excel register with one row per line item. The operator moves from typing to checking, which is where their knowledge actually adds value.

The implementation is deliberately small — five Python modules and one external API — which keeps it maintainable for a college project while still demonstrating the full pipeline of file handling, prompt engineering, structured output parsing and spreadsheet generation.

17. How to Run and Deploy

17.1 Local

```
cd DocEntry-AI
python -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app.py        # http://localhost:8501
```

The key is placed in .env as AI_API_KEY, with AI_BASE_URL pointed at the Gemini OpenAI-compatible endpoint and AI_MODEL set to gemini-3.1-flash-lite. The sidebar shows the model, a masked key fingerprint and a Test connection button.

17.2 Public link

The repository is public on GitHub. Streamlit Community Cloud builds it directly from the main branch using runtime.txt (Python 3.12) and requirements.txt. The same three settings are stored in the app's Secrets panel as TOML, so no key is ever committed. Every push to main triggers an automatic rebuild.

```
AI_API_KEY = "..."
AI_BASE_URL = "https://generativelanguage.googleapis.com/v1beta/openai/"
AI_MODEL = "gemini-3.1-flash-lite"
```

References

1. Streamlit documentation — https://docs.streamlit.io
2. Google AI Studio and Gemini API — https://ai.google.dev
3. OpenAI Python SDK — https://github.com/openai/openai-python
4. PyMuPDF documentation — https://pymupdf.readthedocs.io
5. python-docx documentation — https://python-docx.readthedocs.io

6. openpyxl documentation — https://openpyxl.readthedocs.io
7. Pillow documentation — https://pillow.readthedocs.io
8. Streamlit Community Cloud — https://share.streamlit.io
9. Project repository — https://github.com/prajwalshinde752-nonveg/DocEntry-AI

Appendix A — Field Schemas

Each document type maps to a list of (key, label) pairs in schemas.py. The keys are used in the prompt and in Excel; the labels are shown to the user.

Document type	Field keys
Tax Invoice	invoice_no, invoice_date, seller_name, seller_gstin, buyer_name, buyer_gstin, buyer_address, place_of_supply, hsn_code, description, quantity, unit, rate, taxable_value, igst_amount, cgst_amount, sgst_amount, total_value, currency
Shipping Bill	shipping_bill_no, shipping_bill_date, exporter_name, iec_code, gstin, invoice_no, invoice_date, port_of_loading, port_of_discharge, vessel_name, hsn_code, description, quantity, unit, fob_value, currency, scheme
Purchase Order	po_no, po_date, buyer_name, supplier_name, hsn_code, description, quantity, unit, rate, total_value, currency, delivery_terms
Bill of Entry	be_no, be_date, importer_name, iec_code, gstin, invoice_no, port_of_loading, port_of_discharge, hsn_code, description, quantity, assessable_value, total_duty, currency
Delivery Challan	challan_no, challan_date, consignor, consignee, hsn_code, description, quantity, unit, value, vehicle_no
Audit columns	source_file, extracted_at (added to every type automatically)